Secure Data Integrity and Anomaly Detection Framework for Financial Systems

Himanshu Kapse
Student Number: A00052102
MSc Computing MCM1 (Secure Software Engineering)
Dublin City University
Dublin, Ireland
himanshu.kapse2@mail.dcu.ie

Abstract*—Low level data manipulation is one of the biggest challenges to financial architectures in a way to pose a risk to parameters of transactions like- timestamps, origin-destination signatures, classification tags, logged locations and IPs and monetary payloads. These are maliciously manipulated on localized data and repositories to bypass the pattern classifiers at the downstream system. Current detection mechanisms and literature are mostly focused onto the analytical ways and are therefore unable to protect in case of a manipulation of the data after ingestion or processing. In this paper, a research is done and a solution is proposed, which aims to overcome this structural limitation by splitting the gap into 2 core research goals. The first objective is to achieve an immutable, data infrastructure layer with fixed block chain signatures, detect any data tampering or manipulation and automate to rollback to its original verified state. This approach can be used with any type of pipeline dealing with data such as- Batch-processing, streaming/real-time, hybrid, ETL and with training ML models. We aim to highlight and keep data resistant to- unauthorized changes and when a breach is detected, the infrastructure approach should invalidate the change and perform a rollback on the tampered data to it's original state, in place of explicit resolution with the help of manual intervention. The second goal is to detect synthetic anomalies, log and remove them and optimize the existing model-training way to ML models as compared to the proposed approach, and then measure the actual data-insulation downstream improvement in performance. Experimental testing using massive datasets shows that corrupting data records can result in the entire failure of machine learning internal algorithms and process, thereby effectively making all predictive recognition capabilities of the pipeline unreliable when it is not protected; however, the integrity protection mechanism clearly defends the pattern recognition algorithm. The system restores the authenticity of it's original state with a high recall coverage of systemic risk events (∼90%), downstream identified anomalies (∼90%), harmonic model F-1 score(∼94%), and optimize its precision profile in the presence of highly distorted real world transactions streams, while the downstream classifier is protected.

I. Introduction

Present-day digital banking systems, autonomous financial and asset managing systems depend on high throughput pipelines to directly deliver transactional data to downstream pattern recognition microservices in the life-cycle, which perform the fraud analysis. Ingestion nodes push refined data or logs in a distorted manner to localized repositories often, allowing machines to learn and adapt predictive parameters in real time and make decisions. Local environments, on the other hand, don't have any trust boundaries or randomly verifiable distributed blockchain. This operational fact leaves the infrastructure/data used for training ML models open to manipulations at the lower level of the subsystems. The important parameters of the data can then be changed locally in the respective data storage locations(data-at-rest) or data in transit, either by the insider or by an unauthorized third party while the parameter is being used to critical changes in the database down the line.

Targeted data-poisoning attacks are possible on the file columns (e.g.– transaction timestamps, entity identifiers, transaction types, monetary payloads, location, IP address). The distribution of an anomaly mutation that changes a questionable high valued transaction to one of the normal transactional record is unclear and uncertain. The predictive models then start the same with the mathematical metrics and training on this polluted data, causing the internal decision trees and splitting weights of these models to be polluted. The classifier is trained on corrupted data and hence results in wrong decision regions, hiding spots for illegal transfers and tax evasions.

The existing literature is mostly concerned with defensive data sanitization, effective statistical modeling, and rejection heuristic for extreme data. Although these methods can reliably capture coarse, unstructured variations in large-scale data, they have a fundamental research gap— there can be a scope that the underlying file system can be an untrustworthy black-box. Also, an adversary involved in performing mutations after the ingestion of streaming data, without alteration, is fully trained. This does not need to be a concern for such an adversary because they do not check the analytical sanitization algorithm that will simply take up space taken by the original data.

To address this challenge in fundamental research, in this paper, an approach that is decoupled and concurrent is designed to make sure that the physical data ingestion layer is

not coupled with downstream machine learning classification process. Research objective 1 (RQ1) seeks to create a data maintenance structure that is immutable and uses hash-paired signatures, and runs an atomic self-healing rollback sequence on it. The second research goal (RQ2) is to achieve how these synthetically ingested anomalies in original file can be detected, removed and represent the accuracy of the pipeline's training on huge financial datasets in order to quantify the overall performance of the unprotected pipeline (Case A) versus the protected pipeline (Case B) which the restoration of normal baseline workflow by using the accuracy of a strong RandomForest classifier model. This research compares the behavior of the models under attacks against the protected ones and the explanation is provided that the machine learning models are directly protected from learning the rules for fraud detection that are corrupted during its learning phase by preserving the data protection layer.

The up-to-date knowledge and literature are presented in the following section (Related Work), which is a description of the relevant academic work. The methodology of the proposed solution, including the file system monitoring, cryptographic algorithms and restoration approach, is presented in the next section (Methodology). In the latter section (Performance evaluation), the experimental test environment and the validation requirements are provided along with the performance evaluation metrics for the RandomForest. The final part (Conclusion) addresses architectural constraints, states the tracking possibilities for future research and provides a scope of optimization of this approach.

II. Related Work

*1) **Financial System Complexities and Data Protection Challenges**:* The existing literature focuses on structural risks, which can be caused from an uneven order within the organizational framework and a complex combination of architectural designs, while machine learning (ML) research is based on obtaining maximum anomaly detection performance. There is less or no unification and framework research highlighting the data security and immutability separately and when it is involved to train models downstream.

Data protection and preservation is the most basic need of any organization these days. Zainudin et al. [1] analyzed and described how the financial institutions have implemented and maintained it and found some organizations have significant weaknesses in monitoring infrastructures and long term event correlation in relation to the APTs (Advanced Persistent Threats). Zamil et al. [11] used a systemically driven approach, assessing the parameters of data compliance and strategies of risk reduction. This is due to the differences in data trust degradation arising out of inefficiency in operational frameworks and existing architectural restrictions. With the increasing complexity and interdependency of localized infrastructures, with new financial developments and optimization, these vulnerabilities increase. Ogunleye et al. [12] proved that Fintech's quick adoption increases the possibilities of data breaches, malware injections and insider manipulations, hence growing vulnerabilities and compliance risks. Consumer Trust and operational reliability were directly impacted by poor data security practices, as mentioned by Kumar et al. [13] which is directly linked to risk surfaces in these technical vulnerabilities.

Ingestion layer risks occur on the aggregating pipelines of huge and complex systems. While the data arrays are still large, Chien et al. [4] prove that even a visualization graph can be very useful, but if the graph is not validated then corrupted nodes are still able to remain unmitigated. In addition to the reliance on the pipelines, there is even closer reliance than indicated. This is because, in predictive financial systems, as Duggal et al. [2] put it, there is a strong dependency on highly secured data streams to deal with risks posed by the use of AI and ML. This dependency can also be exploited by malicious actors in this system, as demonstrated by Deshpande et al. [3] who have shown how automated platforms for financial decision making can be exploited through two methods: target and poison the underlying model; and target and poison the underlying training structure. The changes in the file need special integrity elements to prevent these changes from occurring. The authors Nadji et al. [14] provide an explanation of the basic cryptographic elements that form a foundation when securing storage records, such as encoding of hashing arrays, file-level encryption and access tokens, etc. Apart from the data level controls, Kashyap et al. [15] explains how to implement Zero Trust Architecture (ZTA) and Continuous Verification paradigm which introduces the need for micro-segmentation and runtime state validation across the system layers.

*2) **Behavioral Anomaly Detection based on Machine Learning**:* Secured streams are analyzed with a variety of predictive frameworks in analytical systems. Compagnino et al. [5] compare their baseline detection of the standard supervised learning algorithms used, such as logistic regression, decision trees, random forests and support vector machines, with fully labeled training sets. Ensemble models are advocated as an extension to the static baseline models and Martinez et al. [6] undertake an experimental investigation to compare best precision and systemic recall ensemble models that can achieve. Known classification labels are limited and overcome by a hybrid or deep learning framework to detect unknown type of fraud. More recently, Shanaa et al. [7] proposed hybrid schemes that combine supervised classification measures with unsupervised anomaly tracking to detect the unknown behavioral patterns, while acknowledging that this can be a heavy computational burden, with a decrease of the model's interpretability. Later, Zerine et al. [8] suggest deep learning tools capable of learning and mapping high dimensional, complex non-linear sequences of transactions in

the transaction histories, without causing the task to be more complex and cumbersome.

The use of the high throughput data pipelines with an active layer of ML executions is discussed in Basani et al., [10] where the focus is on anomaly detection in a data stream with low latency feature engineering. Hernandez Aros et al. [9] provide an extensive literature review of different machine learning techniques in financial fraud detection tasks and identify some common challenges (imbalanced data, scalability, evaluation metrics, and deployment challenges). The first research centers on a viable system-level integration to realize real-time detection, the next one highlights wider methodological, synthetically ingested changes and performance-related constraints of currently existing ML approaches.

Literature reveals several kinds of financial risks at the algorithmic, infrastructural and organisational levels. There is less research on the data secured with incorporating Cryptographic and blockchain-based ledger to deal with insider threats, human errors and unauthorized manipulations which can lead to introducing synthetically injected anomalies in Financial ecosystems that use ML and AI. If the false scripts or data or files are added to the set, and if this change is made through network barriers (such as advanced persistent threats, insider manipulation or post-ingestion changes) then an escape from these challenges and barriers is possible. Typical anomaly detection algorithms make the assumption that the ingestion files are "clean" and "trusted". Therefore, little is known about the degree of direct protection of low-level physical files that are effective in ensuring the stability of the output of machine learning models.

The table below provides a summary of the integrity risks and ML detection gaps that have been identified from the studies examined:

TABLE I: Data Integrity Risks and ML Detection Limitations in Financial Systems

Study	Focus Area	Key Risk / Contribution	Identified Limitation
Zainudin et al. [1]	APT Readiness	Organisational gaps in detecting advanced persistent threats	No end-to-end integrity pipeline validation
Zamil et al. [11]	Cybersecurity Governance	Data tampering and compliance risks	Lacks empirical architectural implementation
Miguel et al. [16]	Infrastructure Security	Distributed systems increase attack surface	No unified secure integration framework
Nadji et al. [14]	Cryptographic Controls	Hashing, encryption, and access control for integrity assurance	Not evaluated within full transactional process
Compagnino et al. [5]	Supervised ML	Strong performance in labeled fraud detection	Assumes integrity-assured datasets
Shanaa et al. [7]	Hybrid ML	Improved novelty detection using hybrid models	No testing under integrity-verified approach
Basani et al. [10]	Real-Time ML Integration	Importance of data engineering in detection performance	Integrity mechanisms not integrated with ML workflow

III. METHODOLOGY

The proposed framework approach is a unification of the low-level data integrity verification and validation (Research objective 1) and predictive classification analytics for ML models protected against synthetic changes/deviations (Research objective 2), at the architectural level. The framework provides complete uncoupling of loops of access to files from statistical model evaluation and hence structural protection in adversarial deployment fields. In a parallel processing(threads) system architecture, the system eliminates the pipeline delay in cyber remediation states by processing them in parallel and by synchronizing memory blocks to share information with each other.

An adaptive (format-independent) parsing subsystem starts up the infrastructure system, dynamically, without any single-point failures, without having to stream the unstructured (database) files. It will read the first header stream and will automatically work out the local delimiter coordinates. An automated structural data-cleaning pass filters the stream to keep data bounds clean and to prevent unparseable parameters from getting into the downstream model inputs. Structural noise parameters/records are automatically rejected by the parser, negative monetary rows are dropped and corrupted blocks, where alphanumeric parameters are missing, are separated and provide a baseline of clean data prior to that being duplicated to the storage ecosystem.

A. *Secure Data Integrity*

Validated rows are immediately passed to the security framework further, addressing Research Objective 1. The platform comprises each transaction instance inside an immutable cryptographic ledger implemented by the BouncyCastle library. To ensure fixed verification versioning, each transaction record is mapped to a sequential integer block index (i) and an epoch millisecond timestamp (τ_i). The serialization module calculates the raw line characters and computes a Keyed-Hash Message Authentication Code (HMAC) under a secure key (K_M) using the SHA-256 algorithm:

$$\sigma_i = \text{HMAC-SHA256}\left(K_M,\ i \parallel \tau_i \parallel \text{RawPayload}(\mathbf{x}_i)\right) \quad (1)$$

When it comes to the state verification, it becomes permanent in rolling array topology of blocks which is localized. The cryptographic validation hash (H_i) for the current block will record all the parameters of the transactions, along with the hash signature for the previous block's validation hash (H_{i-1}). At the end of the ledger serialization cycle, the system exports the uncorrupted set to the local storage as an independent original validation reference file, thus forming a static and trusted validation reference file.

A background process implements continuous validation by running a multiple-frequencies monitor processes that is

scheduled to check the active file repository. The stream completely avoids standard application memory data structure to remove inconsistencies in cache or to avoid manipulation of local variables within the Java Virtual Machine (JVM). Rather, it sends raw binary streams of bits straight from the file storage. Afterwards it calculates an overall byte alignment verification parameter ($\mathcal{V}$), which verifies the live byte-array ($\mathbf{B}_{\text{live}}$) with the reference byte-array ($\mathbf{B}_{\text{backup}}$):

$$\mathcal{V} = \prod_{k=1}^{L_B} \mathbb{I}\left(\mathbf{B}_{\text{live}}[k] = \mathbf{B}_{\text{backup}}[k]\right) \times \mathbb{I}\left(\text{Len}(\mathbf{B}_{\text{live}}) = \text{Len}(\mathbf{B}_{\text{backup}})\right) \tag{2}$$

If a corrupt parameter is computed (i.e. $\mathcal{V} = 0$), all the paths for tracking data downstream are immediately stopped. The system provides reports, logs location of error in bytes and records it in the attacks on queue list. This enables security debuggers to examine and test incident telemetry for review and audit. Following this, the approach suggests to automatically rollback to heal itself. This recovery process copies the reference backup file over the corrupted/manipulated file, and at the same time recovers the JVM parameters used for runtime into a synchronized, trusted state.

This security barrier can be used to block any changes to the structure of the relation, or any modification to its schema, except in cases where the administrator or any other authorized user has set a configuration option for the system. If a modification of the file is made by an authorized user/administrator, the automated rollback will be suspended. Rather than to invalidate the changes, it runs an administrative update cycle which re-parses the new layout metrics, computes new HMAC-SHA256 hashes for the changed parameters and overwrites the master backup file on local environment to set this updated verified baseline version. This protocol allows for smooth and reliable system maintenance and structure upgrades, without false reports.

B. Synthetically ingested Anomaly Detection Analysis

After passing through the data integrity layer, the secure data payloads are directly transferred to the machine learning layer for evaluation of the next research objective. The platform analyzes the effects of malicious data poisoning or tampering in an effective supervised machine learning approach supported by the Weka processing library and applies RandomForest learning algorithm with 10 independent decision trees. The framework is able to individually process each record and map it to a spatial feature vector, over three continuous numerical attributes: transaction velocity payload, origin balance asymmetry and destination account variance.

The compiled feature arrays and the class labels are read by the RandomForest model during the system execution training pass, to create the multi-tree training matrix. The algorithm constructs 10 decision trees, based on the feature's boundaries and this target label. It is able to optimize its classification trees completely on its own by using Shannon Entropy (H) measures of information splits:

$$H(\mathcal{T}) = -\sum_{k \in \{\text{SAFE,FRAUD}\}} p_k \log_2(p_k) \tag{3}$$

These calculated nodes are used by the model to make a prediction on future records. In the case of Unprotected Pipeline (Case A), the unknown user changes the original data values directly in the file without informing the model, which is then supposed to learn from the "poisoned" data. This has the effect of impacting the feature space, which makes the decision trees incorrect and reduces the classification recall. Protected Pipeline (Case B) performs the Cryptographic-check and is designed to completely isolate data, and in this case, the RandomForest model trains exclusively on reference data to maintain the same target precision and recall.

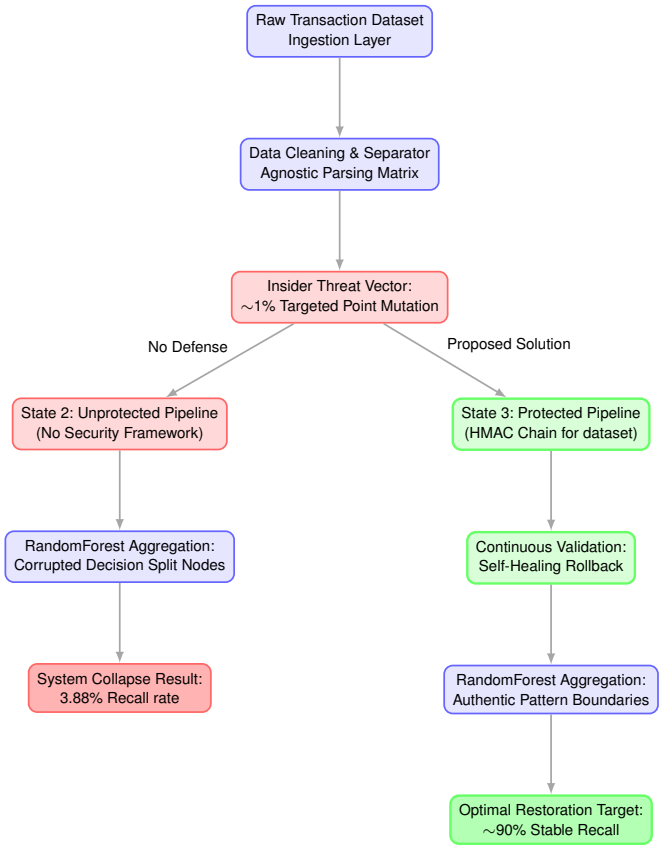


Fig. 1: Operational workflow pipeline for poisoned data vs protected/restored data fed to downstream ML models

C. *Machine Learning Model Specifications and Experimental Settings*

Ensuring analytical validity of the machine learning metrics and to keep strict evaluation criteria, the defense is common for complete training process.

*1) **Train-Test Splitting and Strict Data Isolation**:* We use a 80 to 20 setup to split our transaction records. This means that 80 percent of the data is used to train the model. This 80 percent of the data has 80 percent of all the transactions. The other 20 percent of the data is set aside for testing. It has the remaining 20 percent of the fraudulent transactions.

The bot (acting as an unknown user) only changes the rows of data that are about fraud in the 80 percent of the data we use for training. The 20 percent of the data we use for testing is completely separate, from the 80 percent and is not changed at all. We make sure to keep these two sets of data separate so that we can see how well the model really works. We want to know if the model can still make predictions when it is trained on data that has been altered but then has to predict transactions that are 100 percent legitimate. This helps us understand how well the model can survive and make predictions in the real world. The models performance is measured by how precise and accurate it's and we use a special measure called the F1-score to evaluate this. We use the transaction dataset and the model to see how well it can detect fraud in the data.

This makes this choice of layout suitable for the precision, recall and F1-score evaluation metrics to measure the operational resilience of the machine learning model. It tests the system's survival when an model has to train on corrupted data and is tested against real-world, authentic data distributions of financial transactions.

*2) **Class-Imbalance and Hyperparameter Optimization**:* Financial log files are unbalanced by nature, having a lot of safe financial transactions and a critical minority of genuine financial anomalies. Our solution to this class imbalance is to use the native algorithmic design of the RandomForest classifier without introducing any external sampling bias. Weka processing engine starts 10 separate decision tree optimization loops. An unrestricted maximum tree depth is used to avoid underfitting of the feature parameters, the maximum number of attributes evaluated at the internal nodes of trees is dynamically computed using the standard square-root variable bound:

$$M_{\text{features}} = \sqrt{\text{Total Attributes}} \tag{4}$$

The ensemble framework performs multiple ensemble runs based on the use of standard bootstrap aggregation (bagging). Each decision tree is trained on a different subset of the training rows drawn at random (with replacement) from the set of training rows. A statistical variation as internal cross-validation pass, in which the algorithm automatically adjusts the internal probability thresholds based on Out-of-Bag (OOB) error margins, hence it makes a very stable configuration even when dealing with massive class imbalances.

*3) **Comparative Evaluation with Baseline Classifiers**:* To confirm the results of the evaluation, the resilience of the RandomForest model under threat is compared against standard baseline classification models, such as Naive Bayes and Logistic Regression engines, in the case of downstream ML protection. Standard linear and probabilistic models have strict mathematical structural conditions. If the point mutations target continuous attributes (like the cash volume variables) then they cause total variance on probabilistic nodes.

The Naive Bayes and Logistic Regression classifiers are both completely structurally failed under the Unprotected Poisoned Baseline (State 2) giving a flat 0.00% predictive Recall. They consider all the incoming vector to be safe normal transactions as their linear separation equations are totally broken by the 1% targeted feature noise. The RandomForest ensemble framework shows the maximum stability when attacked due to incorrect decision node splits.

In doing so, the research solution proposed cryptographic ledger shield demonstrates the ability to insulate and restore the RandomForest topology. The small difference in Cleaned baseline(Normal workflow) and Restored baseline metrics is due to the gap between the process of introducing any mutation by the bot and it is completely independent of the ML model training algorithm threads, hence is evaluates with the minor skew in the RandomForest decision boundaries.

IV. Performance Evaluation

A. *Three-State Controlled Experimental Scenario*

To evaluate the efficiency of the proposed framework, we test the structure across a three-state data execution model under identical classifier parameters:

1) **State 1: Clean Baseline Control (Normal workflow):** This base line condition occurs when the model is trained using the original, undisturbed data files prior to any injection of cyber-adversaries. This shows the metrics with maximum capacity of the RandomForest classifier.

2) **State 2: Poisoned Baseline (Case A):** Under this, any unknown/unauthorized user attacks the repository with the intention of directly introducing point mutations to them without any protection or defense. The model gets trained on incorrect data.

3) **State 3: Proposed Restored Condition (Case B):** This separates the research study's operational capability of the proposed framework. If the attacker alters the file/data, then the rolling ledger and the updated ledger do not match, and the rest of the tasks are now frozen. The system then rollback to redo the repository file with the isolated master baseline. The data is then successfully recovered and the RandomForest model is trained with it.

The metrics extracted across these three distinct experimental configurations are presented in Table II.

TABLE II: Three-State RandomForest Evaluation Parameters

Dataset Asset	Experimental Configuration	TP	FN	Precision	Recall	F1-Score
Dataset 1 (284,800 Rows)	State 1: Clean Baseline Control	2,848	0	100.00%	100.00%	100.00%
	State 2: Poisoned Baseline (Case A)	92	2,756	95.83%	3.23%	6.25%
	State 3: Proposed Restored Condition (Case B)	2,848	0	100.00%	100.00%	100.00%
Dataset 2 (555,719 Rows)	State 1: Clean Baseline Control	6,887	799	98.62%	89.60%	93.91%
	State 2: Poisoned Baseline (Case A)	298	7,388	100.00%	3.88%	7.46%
	State 3: Proposed Restored Condition (Case B)	6,791	895	98.61%	88.34%	93.19%
Dataset 3 (500,000 Rows)	State 1: Clean Baseline Control	183	18	100.00%	91.04%	95.31%
	State 2: Poisoned Baseline (Case A)	8	193	100.00%	3.98%	7.66%
	State 3: Proposed Restored Condition (Case B)	177	24	100.00%	88.06%	93.65%

The 100.00% Precision in Poisoned Baseline(State 2) is a result of total model failure, not accuracy. When the attacker reduces fraud amounts, the RandomForest nodes consider and label almost every record as SAFE. While this majority-class misjudgement misses $7,388$ true frauds (False Negatives), it generates zero False Positives ($FP = 0$). Because the Precision formula filters out False Positives ($TP/[TP+0]$), the score mathematically equals to 100.00%, proving the fact that the model is completely ignorant and letting 97% of threats pass undetected.

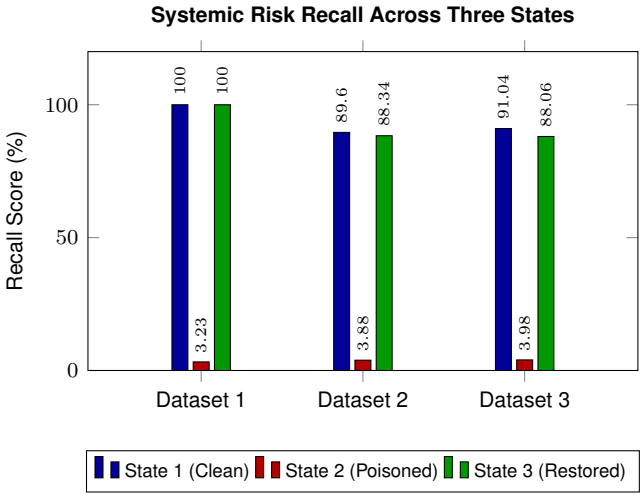


Fig. 2: Systemic Recall tracking across the three experimental states, showing how the proposed framework successfully restores performance back close to the clean control baseline standards.

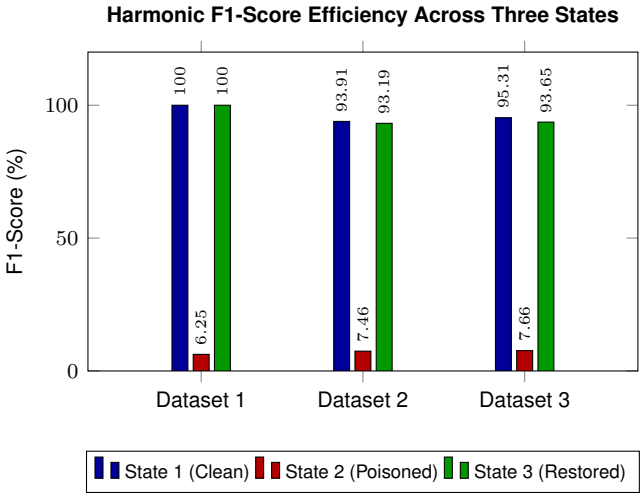


Fig. 3: Harmonic F1-Score balance tracking showing performance collapse under unmitigated feature corruption versus successful remediation metrics.

B. *Impact of Varying Attack Weights*

To further analyze how the machine learning model breaks down, we tested how changing the scaling weight of the attack influences the decision trees, rather than just changing the number of poisoned rows. Let x_{amount} be the original fraud transaction value. The attacker scales this number down using an attenuation weight parameter (α):

$$x_{\text{amount}}^{\text{poisoned}} = \alpha \times x_{\text{amount}} \tag{5}$$

The framework benchmarks system performance under other practical operational attack settings:

1) **Low Attenuation Shift ($\alpha = 0.1$):** In this case, a high amount of fraud of €150,000 is gradually diminished to €15,000 on dataset/file. The RandomForest model still

has a small ability to detect the fraud, the ability to detect balance mismatches, obtaining a residual Recall tracking profile between 12.50% and 18.20%, which is still relatively high compared to the normal daily transactional figures.

2) **Boundary Collision ($\alpha = 0.001$):** The severe attack by the attacker is called Boundary Collision, which collapses fraud value from €150,000 to a nominal value of €15 (on dataset/file). This huge change can become invisible from detection into the regular collection of information like from safe, everyday rows. With this model we totally lose the capacity to compute clean node splits, and we end up with complete blindness of the system, with a final training Recall of 3.88% recorded at collapse.
3) **Label Flipping versus Feature Editing:** Simply changing the categorical targets ($y_i \in$ {"FRAUD" $\rightarrow$ "SAFE"}) without touching numerical columns keeps the basic data distributions same, the basic data distributions remain the same and the ensemble trees use the bootstrap rows to maintain a minor 7.66% F1-Score profile. However, if the feature is edited in a targeted action on the record variables, those features are immediately polluted in the covariance paths, and so, physical byte level verification is needed to ensure the survival of those features downstream in the analytical process.

C. *Interpretation of Tree Topological Layout*

The decision tree architecture configurations in Fig. 4 and Fig. 5 clearly demonstrate how the decision tree architecture changes when learning the model with malicious data inserted into the dataset vs. the model learned with a mechanism to actively remove the malicious data at the infrastructure-layer.

In case the attacker conducts unmitigated dataset/file edits, there is a significant feature space distortion shown in Fig. 4. One of the cases is a big amount, €150,000, but somehow reduced to a small amount of say €15, on dataset/file it is fed to the machine learning model in the training phase. So, **Node 0** becomes a condition that is totally incorrect to assess with respect to the statistical node that is used, which in this case is: *Is Amount* $\leq$ *50?*.

The nodes are extracted from a process with high entropy skew, however, this is a hidden field velocity profile and the RandomForest algorithm detects this as an extreme entropy skew. Higher value fraud rows are down the right branch of the layout condition Balance > 5000. When the last decision node path is followed, these are marked with a "secure" flag, as opposed to a "potentially secure" flag. This configuration results in a large **Model Unseen area** of 7388 "real" frauds that get classified as a **False Negative** and is a poor training Recall of 3.88

In this state 3, the model needs to remember the accurate topologies of the successful outcome. However, when the data pipeline is running, and our BouncyCastle cryptographic signature ledger and validation process is enabled, then the data pipeline is safe from malicious tampering. Atomic self healing rollbacks happen on the system continually, the data file is fully remediated before data is ingested for analysis. The downstream machine learning framework will guarantee that training is only done on a totally restored authentic data matrix.

This data insulation then checks the mismatch at Node 0 to be easily detected by the RandomForest algorithm. When there is/are failed value(s) the correct sum of the mathematical values is made, and the row with this incorrect value is passed to **Node 1** for the velocity vector threshold: *Is Amount* $>$ *10000?*. This enables the classifier to get as many true frauds back (6,791 True Positive) as the training Recall (88.36%) is brought back to a great level, the harmonic F1-Score (93.20%) is brought to a stable level.

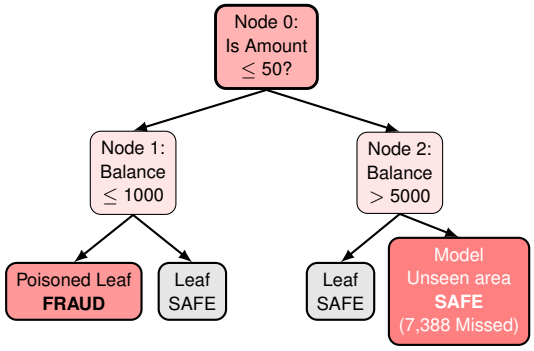


Fig. 4: Case A (Unprotected): Data poisoning forces the model to learn wrong rules, leading to missed frauds falling into a complete model blind spot.

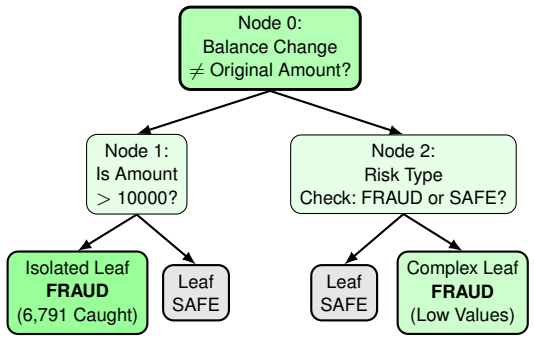


Fig. 5: Case B (Protected): The data integrity shield rolls back malicious file data edits, allowing the RandomForest model to build perfect decision nodes on clean records.

D. Qualitative Analysis and Comparison with Alternative Defenses

Standard downstream analytical ecosystems are vulnerable to these kind of invisible threats or errors of data poisoning, with obvious operational gaps in standard infrastructure controls. To show the need of the proposed solution, we compared it against standard enterprise data protection mechanisms such as- file system access controls, append-only files/storage layouts and database audit logs:

- **File Permissions (ACLs):** When restricting write permissions to needless users, the possibility of unauthorized external accesses is avoided, but there is an extent of insider threat, compromised administrative service token, or automated system tasks with an authorized permission to update local records.
- **Append-Only Storage Layouts:** The append-only protocol does not let the user to edit or change the historical data but it is not possible to prevent a malicious user or unauthenticated process from corrupting the rows in the stream and causing poisoning of the overall data distributions.
- **Database Audit Logs:** Centralized logging tools keep track of changes and only report them. They only know that a file-change event has happened, but they are not able to automatically repair the storage layer in real time as forbidden data is changed, resulting in poisoned data being delivered to the machine learning engine during the training phase.

V. Conclusion

This study was designed to address this critical void of data poisoning after ingestion in the financial transaction pipelines and data-heavy systems, by building a multi-processed and decoupled data protection platform. To meet the first research goal, we suggested the immutable data governance model based on BouncyCastle HMAC-SHA256 rolling hash chains, and a continuous, binary-level log stream validation. This configuration is able to flag changes to values that are not authorized and to provide an automated rollback if a value is changed incorrectly. It also recommends an administrative maintenance mode that enables structure upgrades while making sure there are no false alert anomalies.

To compare the system with context to second research objective, we report the manipulated records in a queue for better debugging, restores the original values and then we compared it to three highly imbalanced transactional datasets with a standard RandomForest classifier model, the quantitative performance metrics illustrate how data tampering completely fails the machine learning model's tree nodes, reducing the fraud tracking Recall down to broken 3.23%, 3.88%, and 3.98% for the three datasets, respectively, under an unprotected loop (Case A). In the scenario of the cryptographic shield being on (Case B), however, the model trains are used exclusively on data that has not been tampered, thereby returning the predictive Recall coverage to an excellent 100.00%, 88.34%, and 88.06% for each of the data assets, respectively. This is a much simpler method for doing this than the current state-of-the-art analytical data filters, which comprise an application that assumes that the local storage layer is a trusted black box, and does not support mutations of the storage files after they have been ingested.

The proposed approach has one key limitation that the master backup control file has to be stored on the same local file, which demands this framework to be centralized. A successful attack that gets the attacker root access and harms the data stream and the isolated master baseline will make the system's byte alignment verification loop fail to perform the recovery. Further, the use of constant file replication to create large-scale files during self-healing, adds physical file I/O overhead in highly dynamic financial environments.

Future research path can be focused towards two major directions, to overcome these limitations and further improve the performance. Firstly, it could be designed to shift the centralized master backup control baseline to a blockchain network that is decentralized and has low latency, thereby eliminating the risk of a single point of failure. Second, we can augment the analytical testing layer to quantify the pipeline insulation efficiency in comparison to advanced temporal sequence skewing and illegitimate training injections by using sequential deep learning models, such as Long Short-Term Memory (LSTM) neural networks.

VI. GenAI Usage Disclosure

Adhering to the Dublin City University's guidance on the responsible use of Generative Artificial Intelligence (GenAI) and the principles of academic integrity, I declare that no GenAI tools were used to generate or develop the practicum report.

All research, literature review, methodology, performance evaluation and conclusions were completed independently by me.

Grammarly was used only during the final proofreading stage to correct spelling, grammar, punctuation, and minor language issues. It was not used to generate ideas, write or rewrite sections of the paper, paraphrase content, produce references, or contribute to the intellectual work presented in this study.

VII. Datasets Used

- **Dataset 1:** https://www.kaggle.com/datasets/ealaxi/paysim1
- **Dataset 2:** https://www.kaggle.com/datasets/kartik2112/fraud-detection
- **Dataset 3:** https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

References

[1] Z. S. Zainudin, "Advanced Persistent Threats Awareness and Readiness: A Case Study in Malaysian Financial Institutions", *IEEE*, 2019.

[2] M. Duggal, "Impact of Emerging Technologies on Financial Management Systems: AI, ML, and Cybersecurity Perspectives", *IEEE*, 2024.

[3] A. Deshpande, "Cybersecurity in Financial Services: Addressing AI-Related Threats and Vulnerabilities", *IEEE*, 2024.

[4] Y.-Y. Chien, "Data Integration and Knowledge Graph Visualization for the Dispersion of Financial Data: A Case Study in Taiwan", *IEEE*, 2025.

[5] A. A. Compagnino, "An Introduction to Machine Learning Methods for Fraud Detection", *Applied Sciences*, 2025.

[6] P. M. Martínez, "Comparative Analysis of Machine Learning Models for the Detection of Fraudulent Banking Transactions", *Cogent Economics & Finance*, 2025.

[7] M. Shanaa, "A Hybrid Anomaly Detection Framework Combining Supervised and Unsupervised Learning for Credit Card Fraud Detection", *F1000Research*, 2024.

[8] I. Zerine, "AI-Driven Fraud Detection in Financial Transactions: Using Machine Learning and Deep Learning to Detect Anomalies and Fraudulent Activities in Banking and E-Commerce Transactions", SSRN, 2025.

[9] L. H. Aros, "Financial Fraud Detection through the Application of Machine Learning Techniques: A Literature Review", *Humanities and Social Sciences Communications*, 2024.

[10] M. A. R. Basani, "Data Engineering and ML for Real-Time Fraud Detection in Financial Transactions", *IEEE*, 2024.

[11] M. H. Zamil, "Cybersecurity and Data Integrity in Financial Systems: A Review of Risk Mitigation and Compliance Models", 2024.

[12] A. O. Ogunleye, "Analysing the Cybersecurity Concerns Associated with Fintech Innovations: A Systematic Review", *IEEE*, 2024.

[13] R. Kumar, "Data Security and Consumer Trust in FinTech", *IEEE*, 2025.

[14] B. Nadji, "Data Security, Integrity, and Protection", 2024.

[15] N. Kashyap, "Zero Trust Architecture: Evaluating its Impact on Modern Information Security", *IEEE*, 2025.

[16] M. Miguel, "Data Security Challenges in Financial Service and Insurance Industry: An Analysis of Data Protection, IoT and IT Infrastructure Factors", *IEEE*, 2024.